



## **Cloud Computing Lab 13**

**Submitted To: Sir Waqas Saleem & Sie Shoaib**  
**Submitted By: Hamail Fatima**  
**Roll No: 2023-BSE-023**  
**Section: 5(A)**

**Course Name: Cloud Computing**

Lab 12 repo link: [https://github.com/Hamail-maker/CC\\_Hamail-Fatima\\_2023-BSE-023\\_Lab13.git](https://github.com/Hamail-maker/CC_Hamail-Fatima_2023-BSE-023_Lab13.git)

## Lab # 13

### Task 0 Lab Setup (Codespace & GH CLI)

All actions below should be executed inside the Codespace shell.

Create Codespace & connect:

# create or open codespace via GH CLI (example)

gh repo create CC\_<YourName>\_<YourRollNumber>/Lab13 --public

gh codespace create --repo <user\_name>/Lab13

gh codespace list

gh codespace ssh -c <your\_codespace\_name>

- **Save screenshot as:** task0\_codespace\_create\_and\_list.png – output showing repo creation/codespace list.

```
PS C:\Users\admin\CC_Hamail-Fatima_2023-BSE-023_Lab13> gh repo create Hamail-maker/CC_Hamail-Fatima_2023-BSE-023_Lab13 --public --add-readme
✓ Created repository Hamail-maker/CC_Hamail-Fatima_2023-BSE-023_Lab13 on github.com
https://github.com/Hamail-maker/CC_Hamail-Fatima_2023-BSE-023_Lab13
PS C:\Users\admin\CC_Hamail-Fatima_2023-BSE-023_Lab13> gh codespace create --repo Hamail-maker/CC_Hamail-Fatima_2023-BSE-023_Lab13
✓ Codespaces usage for this repository is paid for by Hamail-maker
? Choose Machine Type: 2 cores, 8 GB RAM, 32 GB storage
automatic-capybara-pj6xu6vxo6992rq6q
PS C:\Users\admin\CC_Hamail-Fatima_2023-BSE-023_Lab13> gh codespace list
```

| NAME                                       | DISPLAY NAME              | REPOSITORY                                    | BRANCH | STATE     | CREATED AT             |
|--------------------------------------------|---------------------------|-----------------------------------------------|--------|-----------|------------------------|
| potential-eureka-q7xw45r4g5vhpq            | potential eureka          | Hamail-maker/my-lab-repo                      | main*  | Shutdown  | about 24 days ago      |
| reimagined-orbit-x54jgg6pjag5hrj9p         | reimagined orbit          | Hamail-maker/CC_hamail-maker_023_Lab11        | main*  | Shutdown  | about 20 days ago      |
| redesigned-space-telegram-v65xrj4rx7r5hv56 | redesigned space telegram | Hamail-maker/cc-hamail-fatima-023             | main   | Shutdown  | about 7 days ago       |
| ideal-waddle-jjpx7p67xq76fj659             | ideal waddle              | Hamail-maker/CC_Hamail-Fatima_2023-BSE-023    | main   | Shutdown  | about 7 days ago       |
| humble-space-giggle-jjpx7p67xq7j2pc74      | humble space giggle       | Hamail-maker/CC_Hamail-Fatima_2023-BSE-023... | main   | Shutdown  | about 7 days ago       |
| automatic-capybara-pj6xu6vxo6992rq6q       | automatic capybara        | Hamail-maker/CC_Hamail-Fatima_2023-BSE-023... | main   | Available | less than a minute ago |

```
PS C:\Users\admin\CC_Hamail-Fatima_2023-BSE-023_Lab13>
```

- **Save screenshot as:** task0\_codespace\_ssh\_connected.png – terminal inside the Codespace shell after ssh.

```
PS C:\Users\admin\CC_Hamail-Fatima_2023-BSE-023_Lab13> gh codespace ssh -c automatic-capybara-pj6xu6vxo6992rq6q
Welcome to Ubuntu 24.04.3 LTS (GNU/Linux 6.8.0-1030-azure x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

@Hamail-maker →/workspaces/CC_Hamail-Fatima_2023-BSE-023_Lab13 (main) $
```

### Task 1 – Create IAM Group and Output Details

In this task, you will create an IAM group named "developers" and output its details.

1. Create the initial project structure:

mkdir -p ~/Lab13

cd ~/Lab13

- **Save screenshot as:** task1\_project\_directory.png – terminal showing directory creation.

```
@Hamail-maker →/workspaces/CC_Hamail-Fatima_2023-BSE-023_Lab13 (main) $ mkdir -p ~/Lab13
cd ~/Lab13
@Hamail-maker →~/Lab13 $
```

2. Create the main Terraform file:

touch main.tf

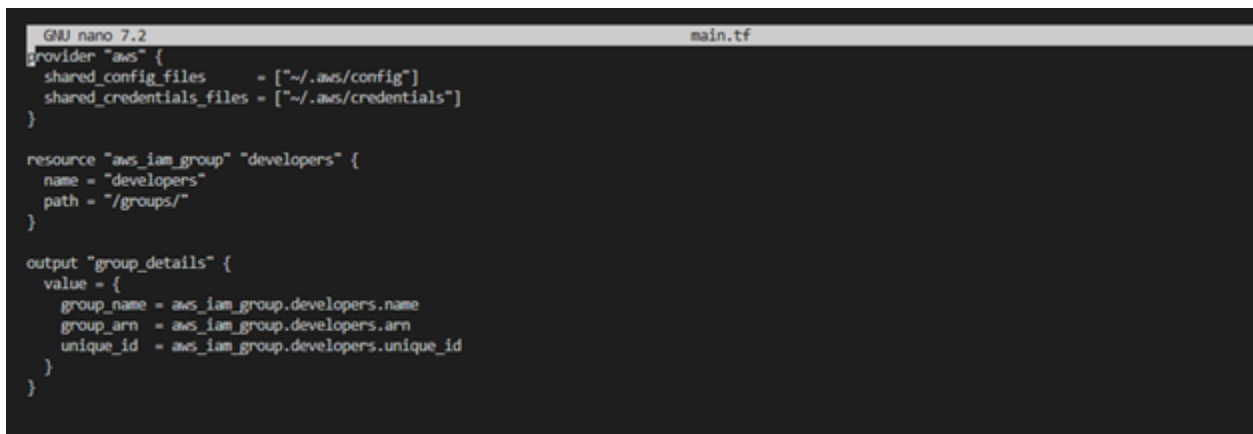
- **Save screenshot as:** task1\_file\_created.png – terminal showing file creation.

```
@Hamail-maker →~/Lab13 $ touch main.tf
@Hamail-maker →~/Lab13 $
```

### 3. Create main.tf with AWS provider configuration:

```
provider "aws" {  
  shared_config_files    = ["~/.aws/config"]  
  shared_credentials_files = ["~/.aws/credentials"]  
}  
  
resource "aws_iam_group" "developers" {  
  name = "developers"  
  path = "/groups/"  
}  
  
output "group_details" {  
  value = {  
    group_name = aws_iam_group.developers.name  
    group_arn  = aws_iam_group.developers.arn  
    unique_id  = aws_iam_group.developers.unique_id  
  }  
}
```

- **Save screenshot as:** task1\_main\_tf. png – content of main.tf file.

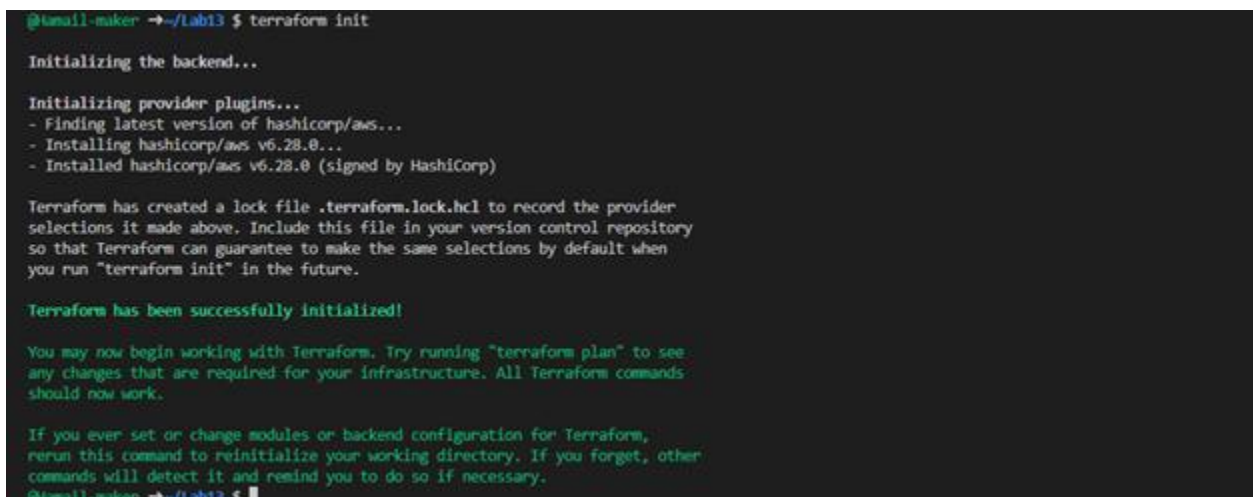
A screenshot of a terminal window with a dark background. The title bar shows 'GNU nano 7.2' on the left and 'main.tf' on the right. The terminal displays the Terraform configuration code from the previous block, formatted with syntax highlighting: 'provider' is in blue, 'resource' is in red, 'output' is in green, and strings are in yellow. The code is enclosed in curly braces and uses indentation for nested blocks.

```
GNU nano 7.2 main.tf  
provider "aws" {  
  shared_config_files    = ["~/.aws/config"]  
  shared_credentials_files = ["~/.aws/credentials"]  
}  
  
resource "aws_iam_group" "developers" {  
  name = "developers"  
  path = "/groups/"  
}  
  
output "group_details" {  
  value = {  
    group_name = aws_iam_group.developers.name  
    group_arn  = aws_iam_group.developers.arn  
    unique_id  = aws_iam_group.developers.unique_id  
  }  
}
```

### 4. Initialize Terraform:

terraform init

- **Save screenshot as:** task1\_terraform\_init.png – terraform init output.

A screenshot of a terminal window with a dark background. The prompt is '@tmail-maker →~/lab13 \$'. The output of the 'terraform init' command is shown in green text. It includes messages about initializing the backend, installing provider plugins (finding and installing hashicorp/aws v6.28.0), creating a lock file, and a final confirmation that Terraform has been successfully initialized. It also provides instructions on how to use Terraform and how to reinitialize it if needed.

```
@tmail-maker →~/lab13 $ terraform init  
  
Initializing the backend...  
  
Initializing provider plugins...  
- Finding latest version of hashicorp/aws...  
- Installing hashicorp/aws v6.28.0...  
- Installed hashicorp/aws v6.28.0 (signed by HashiCorp)  
  
Terraform has created a lock file .terraform.lock.hcl to record the provider  
selections it made above. Include this file in your version control repository  
so that Terraform can guarantee to make the same selections by default when  
you run "terraform init" in the future.  
  
Terraform has been successfully initialized!  
  
You may now begin working with Terraform. Try running "terraform plan" to see  
any changes that are required for your infrastructure. All Terraform commands  
should now work.  
  
If you ever set or change modules or backend configuration for Terraform,  
rerun this command to reinitialize your working directory. If you forget, other  
commands will detect it and remind you to do so if necessary.  
@tmail-maker →~/lab13 $
```

5. Apply the configuration:

terraform apply -auto-approve

- **Save screenshot as:** task1\_terraform\_apply.png – terraform apply output showing group created.

```
Terraform will perform the following actions:

# aws_iam_group.developers will be created
+ resource "aws_iam_group" "developers" {
  + arn      = (known after apply)
  + id       = (known after apply)
  + name     = "developers"
  + path     = "/groups/"
  + unique_id = (known after apply)
}

Plan: 1 to add, 0 to change, 0 to destroy.

Changes to Outputs:
+ group_details = {
  + group_arn = (known after apply)
  + group_name = "developers"
  + unique_id = (known after apply)
}
aws_iam_group.developers: Creating...
aws_iam_group.developers: Creation complete after 2s [id=developers]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.

Outputs:

group_details = {
  "group_arn" = "arn:aws:iam::880794402980:group/groups/developers"
  "group_name" = "developers"
  "unique_id" = "AGPA42E3QACSHDTIJ2EOP"
}
```

6. Display the output:

terraform output

- **Save screenshot as:** task1\_terraform\_output.png – terraform output showing group details.

```
aws_iam_group.developers: Creating...
aws_iam_group.developers: Creation complete after 2s [id=developers]

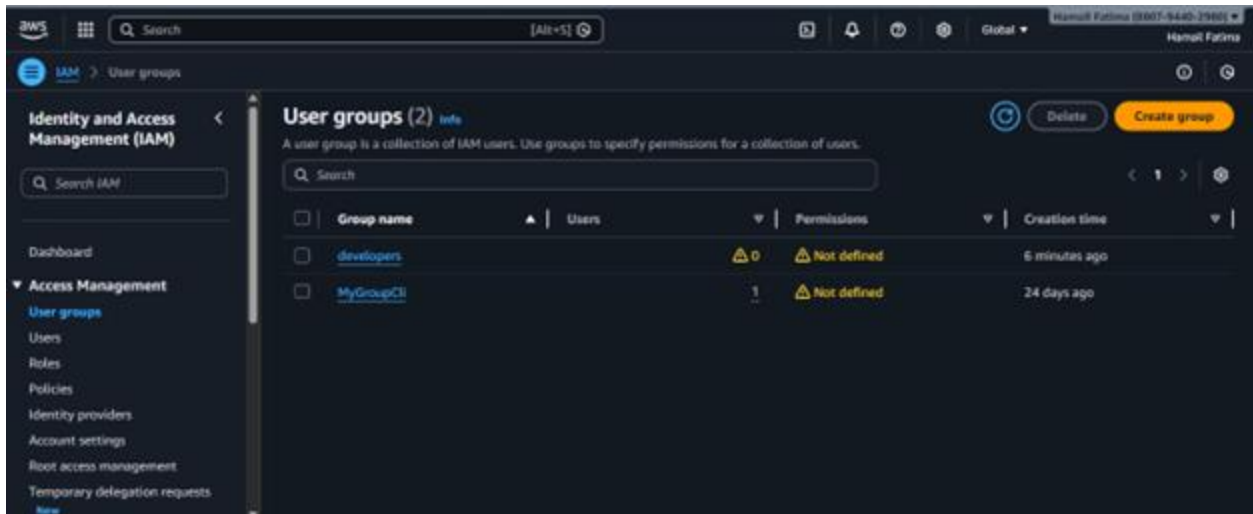
Apply complete! Resources: 1 added, 0 changed, 0 destroyed.

Outputs:

group_details = {
  "group_arn" = "arn:aws:iam::880794402980:group/groups/developers"
  "group_name" = "developers"
  "unique_id" = "AGPA42E3QACSHDTIJ2EOP"
}
@Hamail-maker → ~/Lab13 $ terraform output
group_details = {
  "group_arn" = "arn:aws:iam::880794402980:group/groups/developers"
  "group_name" = "developers"
  "unique_id" = "AGPA42E3QACSHDTIJ2EOP"
}
```

7. Verify the group in AWS Console:

- Navigate to IAM → Groups in AWS Console
- **Save screenshot as:**



– AWS Console showing the developers group.

## Task 2 – Create IAM User with Group Membership

In this task, you will create an IAM user named "loadbalancer" and add it to the developers group.

1. Update main.tf to add the IAM user resource:

```
provider "aws" {
  shared_config_files    = ["~/.aws/config"]
  shared_credentials_files = ["~/.aws/credentials"]
}

resource "aws_iam_group" "developers" {
  name = "developers"
  path = "/groups/"
}

output "group_details" {
  value = {
    group_name = aws_iam_group.developers.name
    group_arn  = aws_iam_group.developers.arn
    unique_id  = aws_iam_group.developers.unique_id
  }
}

resource "aws_iam_user" "lb" {
  name = "loadbalancer"
  path = "/users/"
  force_destroy = true
  tags = {
    DisplayName = "Load Balancer"
  }
}

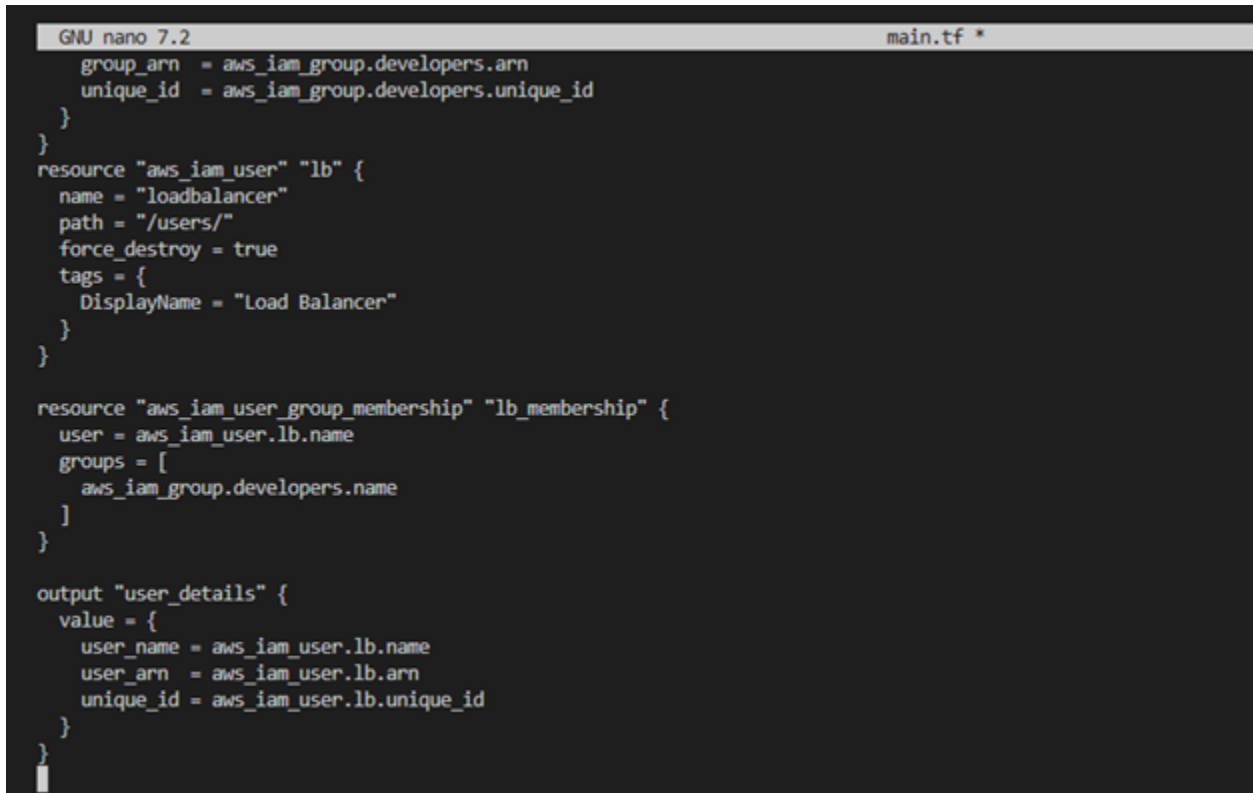
resource "aws_iam_user_group_membership" "lb_membership" {
  user = aws_iam_user.lb.name
  groups = [
    aws_iam_group.developers.name
  ]
}
```

```

output "user_details" {
  value = {
    user_name = aws_iam_user.lb.name
    user_arn  = aws_iam_user.lb.arn
    unique_id = aws_iam_user.lb.unique_id
  }
}

```

- **Save screenshot as:**



```

GNU nano 7.2 main.tf *
    group_arn = aws_iam_group.developers.arn
    unique_id = aws_iam_group.developers.unique_id
  }
}
resource "aws_iam_user" "lb" {
  name = "loadbalancer"
  path = "/users/"
  force_destroy = true
  tags = {
    DisplayName = "Load Balancer"
  }
}

resource "aws_iam_user_group_membership" "lb_membership" {
  user = aws_iam_user.lb.name
  groups = [
    aws_iam_group.developers.name
  ]
}

output "user_details" {
  value = {
    user_name = aws_iam_user.lb.name
    user_arn  = aws_iam_user.lb.arn
    unique_id = aws_iam_user.lb.unique_id
  }
}

```

– updated main.tf with user resources.

## 2. Apply the configuration:

terraform apply -auto-approve

- **Save screenshot as:** task2\_terraform\_apply.png – terraform apply output showing user created.

```
@Hamail-maker →~/Lab13 $ terraform apply -auto-approve
aws_iam_group.developers: Refreshing state... [id=developers]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
+ create

Terraform will perform the following actions:

# aws_iam_user.lb will be created
+ resource "aws_iam_user" "lb" {
  + arn          = (known after apply)
  + force_destroy = true
  + id           = (known after apply)
  + name         = "loadbalancer"
  + path         = "/users/"
  + tags         = {
    + "DisplayName" = "Load Balancer"
  }
  + tags_all     = {
    + "DisplayName" = "Load Balancer"
  }
  + unique_id    = (known after apply)
}

# aws_iam_user_group_membership.lb_membership will be created
+ resource "aws_iam_user_group_membership" "lb_membership" {
  + groups = [
    + "developers",
  ]
}
```

### 3. Display the outputs:

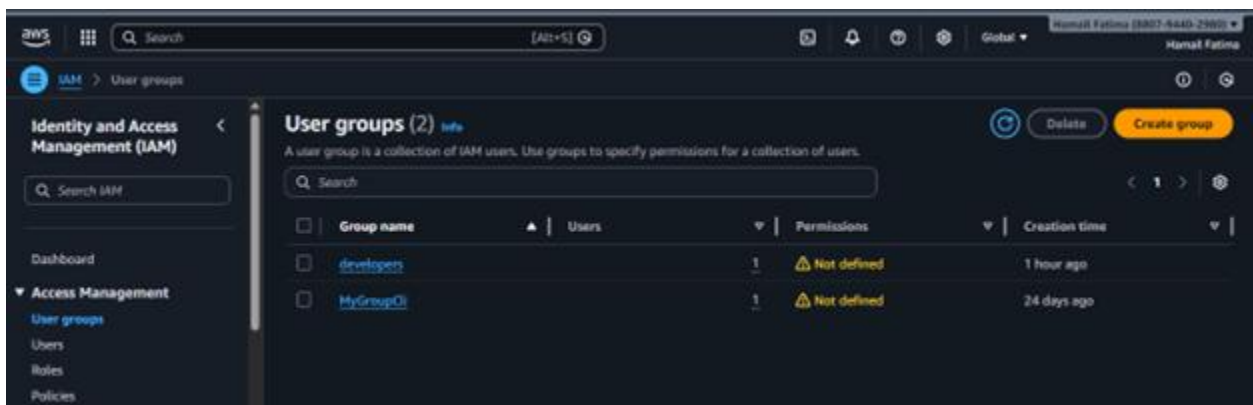
terraform output

- **Save screenshot as:** task2\_terraform\_output.png – terraform output showing user and group details.

```
@Hamail-maker →~/Lab13 $ terraform output
group_details = {
  "group_arn" = "arn:aws:iam::880794402980:group/groups/developers"
  "group_name" = "developers"
  "unique_id" = "AGPA42E3QACSHDTI32E0P"
}
user_details = {
  "unique_id" = "AIDA42E3QACSL6W5GHQDT"
  "user_arn" = "arn:aws:iam::880794402980:user/users/loadbalancer"
  "user_name" = "loadbalancer"
}
```

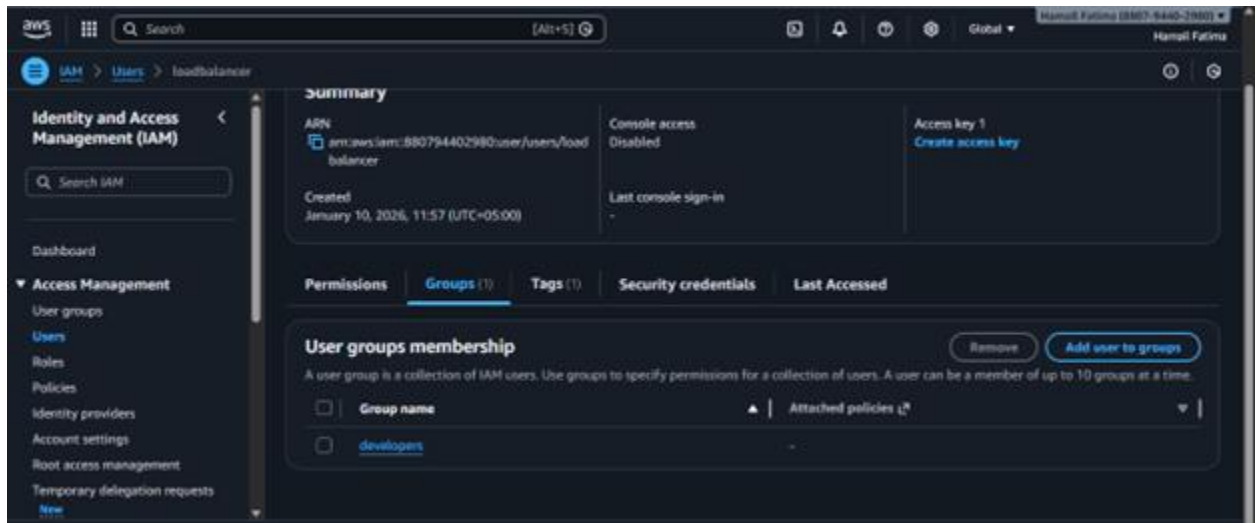
### 4. Verify the user in AWS Console:

- Navigate to IAM → Users in AWS Console
- Click on "loadbalancer" user
- Check the "Groups" tab
- **Save screenshot as:** task2\_aws\_console\_user.png – AWS Console showing the loadbalancer user.





- **Save screenshot as:** task2\_aws\_console\_user\_groups.png – AWS Console showing user's group membership.



### Task 3 – Attach Policies to IAM Group

In this task, you will attach AWS managed policies (AmazonEC2FullAccess and IAMUserChangePassword) to the developers group.

1. Update main.tf to add policy attachments:

```
provider "aws" {
  shared_config_files    = ["~/.aws/config"]
  shared_credentials_files = ["~/.aws/credentials"]
}

resource "aws_iam_group" "developers" {
  name = "developers"
  path = "/groups/"
}

output "group_details" {
  value = {
    group_name = aws_iam_group.developers.name
    group_arn  = aws_iam_group.developers.arn
    unique_id  = aws_iam_group.developers.unique_id
  }
}

resource "aws_iam_user" "lb" {
  name = "loadbalancer"
  path = "/users/"
  force_destroy = true
  tags = {
    DisplayName = "Load Balancer"
  }
}

resource "aws_iam_user_group_membership" "lb_membership" {
  user = aws_iam_user.lb.name
  groups = [
    aws_iam_group.developers.name
  ]
}
```



```

}

output "user_details" {
  value = {
    user_name = aws_iam_user.lb.name
    user_arn  = aws_iam_user.lb.arn
    unique_id = aws_iam_user.lb.unique_id
  }
}

resource "aws_iam_group_policy_attachment" "developer_ec2_fullaccess" {
  group = aws_iam_group.developers.name
  policy_arn = "arn:aws:iam::aws:policy/AmazonEC2FullAccess"
}

resource "aws_iam_group_policy_attachment" "change_password" {
  group = aws_iam_group.developers.name
  policy_arn = "arn:aws:iam::aws:policy/IAMUserChangePassword"
}

```

- Save screenshot as: \

```

# Task 3: Attach AWS managed policies to developers group
resource "aws_iam_group_policy_attachment" "developer_ec2_fullaccess" {
  group = aws_iam_group.developers.name
  policy_arn = "arn:aws:iam::aws:policy/AmazonEC2FullAccess"
}

resource "aws_iam_group_policy_attachment" "change_password" {
  group = aws_iam_group.developers.name
  policy_arn = "arn:aws:iam::aws:policy/IAMUserChangePassword"
}

```

– updated main.tf with policy attachments.

## 2. Apply the configuration:

terraform apply -auto-approve

- Save screenshot as: task3\_terraform\_apply.png – terraform apply output showing policies attached.

```

+ id      = (known after apply)
+ policy_arn = "arn:aws:iam::aws:policy/IAMUserChangePassword"
}

# aws_iam_group_policy_attachment.developer_ec2_fullaccess will be created
+ resource "aws_iam_group_policy_attachment" "developer_ec2_fullaccess" {
+   group = "developers"
+   id    = (known after apply)
+   policy_arn = "arn:aws:iam::aws:policy/AmazonEC2FullAccess"
}

Plan: 2 to add, 0 to change, 0 to destroy.
aws_iam_group_policy_attachment.change_password: Creating...
aws_iam_group_policy_attachment.developer_ec2_fullaccess: Creating...
aws_iam_group_policy_attachment.change_password: Creation complete after 1s [id-developers-20260110080811142100000001]
aws_iam_group_policy_attachment.developer_ec2_fullaccess: Creation complete after 1s [id-developers-20260110080811160100000002]

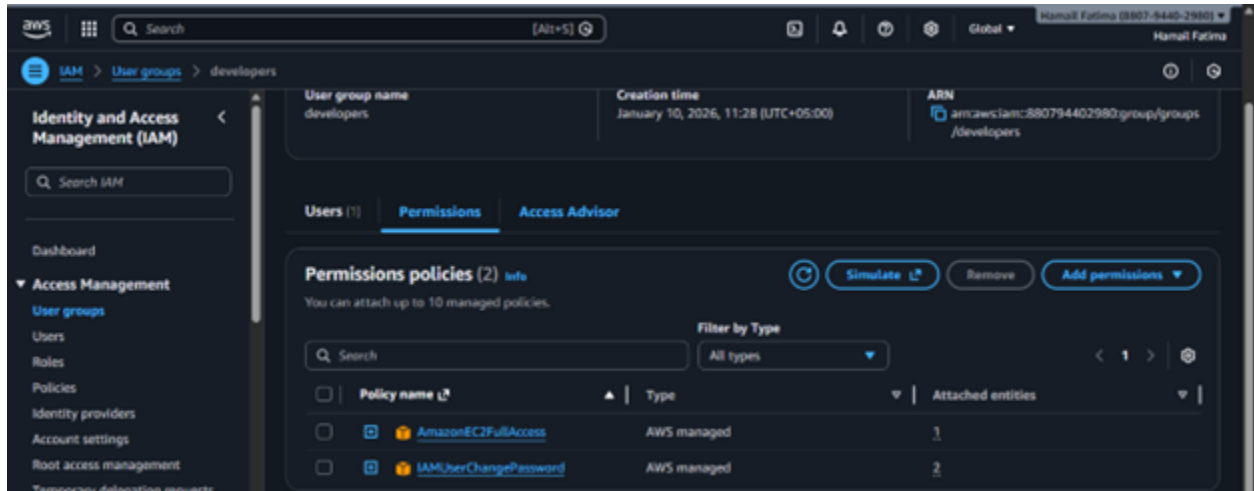
Apply complete! Resources: 2 added, 0 changed, 0 destroyed.

Outputs:

group_details = {
  "group_arn" = "arn:aws:iam::880794482980:group/groups/developers"
  "group_name" = "developers"
  "unique_id" = "AGPM42E3QACSHDTI12EOP"
}
user_details = {
  "unique_id" = "AIDM42E3QACSLG4SGHQT1"
  "user_arn" = "arn:aws:iam::880794482980:user/users/loadbalancer"
  "user_name" = "loadbalancer"
}
@tmail-maker → ~/lab13 $

```

3. Verify policies in AWS Console:
  - Navigate to IAM → Groups → developers
  - Click on "Permissions" tab
  - **Save screenshot as:** task3\_aws\_console\_policies.png – AWS Console showing attached policies.



#### Task 4 – Create Login Profile for IAM User

In this task, you will create a login profile for the loadbalancer user using a bash script and null\_resource provisioner.

1. Create variables.tf file:
 

```
variable "iam_password" {
  description = "Temporary password for the IAM user"
  type        = string
  sensitive   = true
  default     = "1dontKnow"
}
```

- **Save screenshot as:** task4\_variables\_tf.png – content of variables.tf file.



2. Create the bash script create-login-profile.sh:
 

```
#!/usr/bin/env bash
set -euo pipefail
```

```
USERNAME="$1"
PASSWORD="$2"
```

```
# Check if login profile already exists
if aws iam get-login-profile --user-name "$USERNAME" >/dev/null 2>&1; then
  echo "Login profile already exists for $USERNAME. Skipping."
else
  echo "Creating login profile for $USERNAME"
  aws iam create-login-profile \
    --user-name "$USERNAME" \
    --password "$PASSWORD" \
    --password-reset-required
```

fi

- **Save screenshot as:** task4\_create\_login\_script.png – content of create-login-profile.sh.

```
@Hamail-maker →~/Lab13 $ nano create-login-profile.sh
@Hamail-maker →~/Lab13 $ cat create-login-profile.sh
#!/usr/bin/env bash
set -euo pipefail

USERNAME="$1"
PASSWORD="$2"

# Check if login profile already exists
if aws iam get-login-profile --user-name "$USERNAME" >/dev/null 2>&1; then
    echo "Login profile already exists for $USERNAME. Skipping."
else
    echo "Creating login profile for $USERNAME"
    aws iam create-login-profile \
        --user-name "$USERNAME" \
        --password "$PASSWORD" \
        --password-reset-required
fi
@Hamail-maker →~/Lab13 $
```

3. Make the script executable:

chmod +x create-login-profile.sh

- **Save screenshot as:** task4\_chmod\_script.png – terminal showing chmod command.

```
@Hamail-maker →~/Lab13 $ chmod +x create-login-profile.sh
@Hamail-maker →~/Lab13 $
```

4. Update main.tf to add the null\_resource provisioner:

Add this resource after the user creation:

```
resource "null_resource" "create_login_profile" {
  triggers = {
    password_hash = sha256(var.iam_password)
    user          = aws_iam_user.lb.name
  }
}
```

depends\_on = [aws\_iam\_user. lb]

```
provisioner "local-exec" {
  command = "${path.module}/create-login-profile.sh ${aws_iam_user.lb.name}
${var.iam_password}"
}
```

- **Save screenshot as:** task4\_main\_tf\_login\_profile.png – main.tf showing null\_resource.

```
output "user_details" {
  value = {
    user_name = aws_iam_user.lb.name
    user_arn  = aws_iam_user.lb.arn
    unique_id = aws_iam_user.lb.unique_id
  }
}

resource "null_resource" "create_login_profile" {
  triggers = {
    password_hash = sha256(var.iam_password)
    user          = aws_iam_user.lb.name
  }

  depends_on = [aws_iam_user.lb]

  provisioner "local-exec" {
    command = "${path.module}/create-login-profile.sh ${aws_iam_user.lb.name} ${var.iam_password}"
  }
}
```

5. Apply the configuration with a custom password:

terraform apply -auto-approve -var="iam\_password=MySecurePass123!"

- **Save screenshot as:** task4\_terraform\_apply.png – terraform apply output showing login profile creation.

```
@Hmail-maker → ~/Lab13 $ terraform apply -auto-approve -var="iam_password=MySecurePass123!"
aws_iam_user.lb: Refreshing state... [id=loadbalancer]
aws_iam_group.developers: Refreshing state... [id=developers]
aws_iam_group_policy_attachment.change_password: Refreshing state... [id=developers-202601100808111421000000001]
aws_iam_group_policy_attachment.developer_ec2_fullaccess: Refreshing state... [id=developers-202601100808111601000000002]
aws_iam_user_group_membership.lb_membership: Refreshing state... [id=terraform-202601100657391026000000001]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
+ create

Terraform will perform the following actions:

# null_resource.create_login_profile will be created
+ resource "null_resource" "create_login_profile" {
+   id          = (known after apply)
+   triggers = {
+     "password_hash" = (sensitive value)
+     "user"          = "loadbalancer"
+   }
+ }

Plan: 1 to add, 0 to change, 0 to destroy.
null_resource.create_login_profile: Creating...
null_resource.create_login_profile: Provisioning with 'local-exec'...
null_resource.create_login_profile (local-exec): (output suppressed due to sensitive value in config)
```

#### 6. Verify login profile creation:

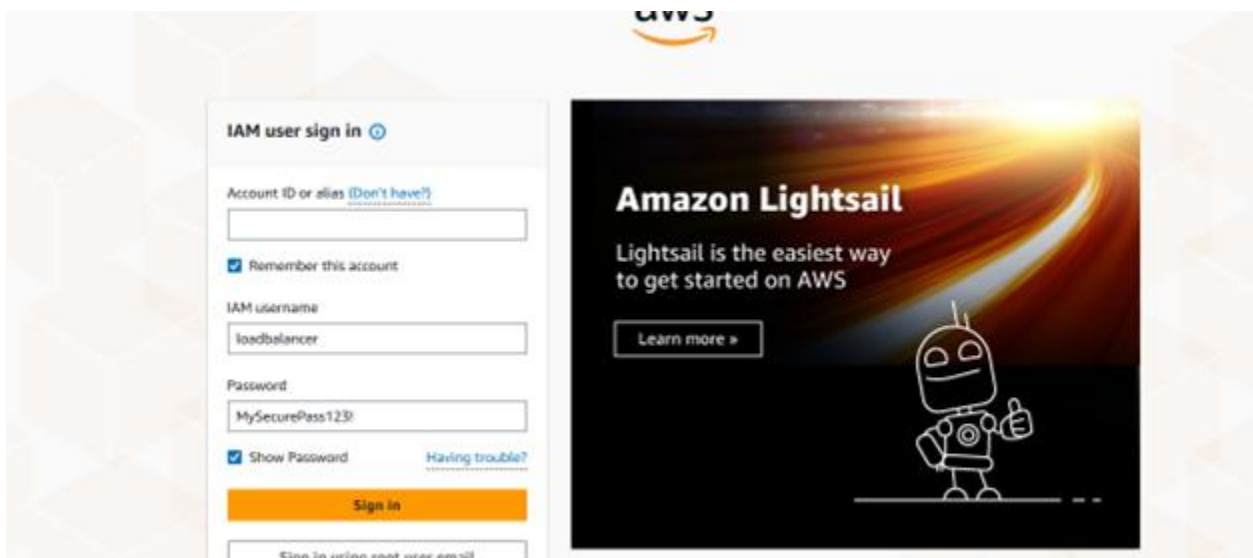
aws iam get-login-profile --user-name loadbalancer

- **Save screenshot as:** task4\_aws\_cli\_verify.png – AWS CLI output showing login profile.

```
@Hmail-maker → ~/Lab13 $ aws iam get-login-profile --user-name loadbalancer
{
  "LoginProfile": {
    "UserName": "loadbalancer",
    "CreateDate": "2026-01-10T08:19:38+00:00",
    "PasswordResetRequired": true
  }
}
```

#### 7. Test login in AWS Console:

- Open AWS Console login page
- Sign in as IAM user with username "loadbalancer" and the password you set
- You should be prompted to change password
- **Save screenshot as:** task4\_aws\_console\_login.png – AWS Console login page with IAM user.



- **Save screenshot as:** task4\_aws\_console\_password\_reset.png – Password reset prompt.

## Password reset ⓘ

Your account **(880794402980)** password has expired or requires a reset.

To continue, please verify your old and set a new password for **loadbalancer** (not you?).

Old Password

☒ Show Password

---

New Password

Confirm New Password

☐ Show Password Matches

**Confirm Password Change**

### Task 5 – Generate Access Keys for IAM User

In this task, you will create access keys for the loadbalancer user and view them in terraform state.

1. Update main.tf to add access key resource and outputs:

Add these resources:

```
resource "aws_iam_access_key" "lb_access_key" {
  user = aws_iam_user.lb.name
}

output "access_key_id" {
  value = aws_iam_access_key.lb_access_key.id
}

output "access_key_secret" {
  value     = aws_iam_access_key.lb_access_key.secret
  sensitive = true
}
```

- **Save screenshot as:** task5\_main\_tf\_access\_keys.png — main.tf showing access key resources.

```
GNU nano 7.2 main.tf *

# Task 3: Attach AWS managed policies to developers group
resource "aws_iam_group_policy_attachment" "developer_ec2_fullaccess" {
  group      = aws_iam_group.developers.name
  policy_arn = "arn:aws:iam::aws:policy/AmazonEC2FullAccess"
}

resource "aws_iam_group_policy_attachment" "change_password" {
  group      = aws_iam_group.developers.name
  policy_arn = "arn:aws:iam::aws:policy/IAMUserChangePassword"
}

# Create Access Key for Load Balancer User
resource "aws_iam_access_key" "lb_access_key" {
  user = aws_iam_user.lb.name
}

# Output the Access Key ID
output "access_key_id" {
  value = aws_iam_access_key.lb_access_key.id
}

# Output the Access Key Secret (sensitive)
output "access_key_secret" {
  value     = aws_iam_access_key.lb_access_key.secret
  sensitive = true
}
}
```

## 2. Apply the configuration:

terraform apply -auto-approve -var="iam\_password=MySecurePass123!"

- **Save screenshot as:** task5\_terraform\_apply.png — terraform apply output showing access key created.

```
+ key_fingerprint      = (known after apply)
+ secret               = (sensitive value)
+ ses_smtpp_password_v4 = (sensitive value)
+ status               = "Active"
+ user                 = "loadbalancer"
}

Plan: 1 to add, 0 to change, 0 to destroy.

Changes to Outputs:
+ access_key_id      = (known after apply)
+ access_key_secret = (sensitive value)
aws_iam_access_key.lb_access_key: Creating...
aws_iam_access_key.lb_access_key: Creation complete after 1s [id-AKIA42E3QACS304PH6W]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.

Outputs:
access_key_id = "AKIA42E3QACS304PH6W"
access_key_secret = <sensitive>
group_details = {
  "group_arn" = "arn:aws:iam::888794402980:group/groups/developers"
  "group_name" = "developers"
  "unique_id" = "AGPA42E3QACS4DTI32E0P"
}
user_details = {
  "unique_id" = "AIDA42E3QACSL6SGHQDT"
  "user_arn" = "arn:aws:iam::888794402980:user/users/loadbalancer"
  "user_name" = "loadbalancer"
}
@tanail-maker →~/lab13 $
```

## 3. Display outputs:

terraform output

- **Save screenshot as:** task5\_terraform\_output.png — terraform output showing access\_key\_id but hidden secret.



```
@iamail-maker → ~/Lab13 $ terraform output
access_key_id = "AKIA42E3QAC5JKWMHY6M"
access_key_secret = <sensitive>
group_details = {
  "group_arn" = "arn:aws:iam::880794402980:group/groups/developers"
  "group_name" = "developers"
  "unique_id" = "AGPA42E3QACSHDTIJ2EOP"
}
user_details = {
  "unique_id" = "AIDA42E3QACSL6W5GHQDT"
  "user_arn" = "arn:aws:iam::880794402980:user/users/loadbalancer"
  "user_name" = "loadbalancer"
}
```

4. View the secret in terraform state:

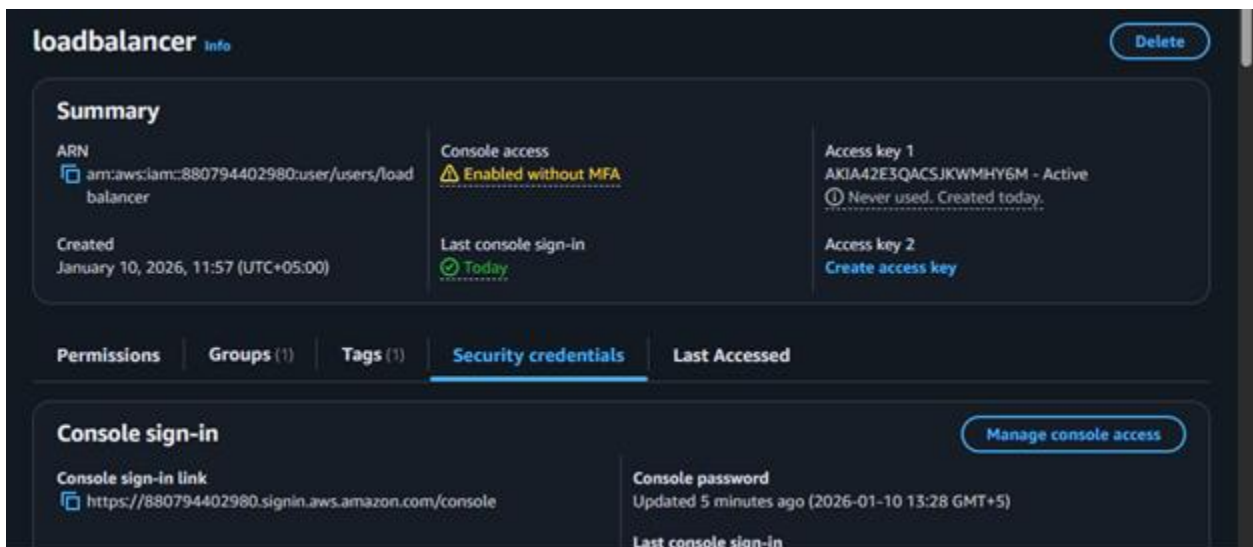
cat terraform.tfstate | grep -A 10 "access\_key\_secret"

- **Save screenshot as:** task5\_tfstate\_secret.png – terraform.tfstate showing access key secret.

```
@iamail-maker → ~/Lab13 $ cat terraform.tfstate | grep -A 10 "access_key_secret"
"access_key_secret": {
  "value": "1IKCd1AWAZG52T1BuPLXonDydEcI6VZZeXk15Mdd",
  "type": "string",
  "sensitive": true
},
"group_details": {
  "value": {
    "group_arn": "arn:aws:iam::880794402980:group/groups/developers",
    "group_name": "developers",
    "unique_id": "AGPA42E3QACSHDTIJ2EOP"
  }
}
```

5. Verify access key in AWS Console:

- Navigate to IAM → Users → loadbalancer → Security credentials
- **Save screenshot as:** task5\_aws\_console\_access\_keys.png – AWS Console showing access keys.



## Task 6 – Implement Terraform Remote State with S3

In this task, you will configure Terraform to use S3 backend for remote state storage.

1. Create S3 bucket in AWS Console:

- Navigate to S3 in AWS Console
- Click "Create bucket"
- Bucket name: myapp-s3-bucket-demo (use a unique name if this is taken)



- Enable versioning
- Keep other settings as default
- Click "Create bucket"
- Save screenshot as: task6\_s3\_bucket\_create.png – S3 bucket creation page.

**Create bucket** [Info](#)

Buckets are containers for data stored in S3.

**General configuration**

**AWS Region**  
Middle East (UAE) me-central-1

**Bucket name** [Info](#)  
myapp-s3-bucket-demo

Bucket names must be 3 to 63 characters and unique within the global namespace. Bucket names must also begin and end with a letter or number. Valid characters are a-z, 0-9, periods (.), and hyphens (-). [Learn more](#) [u](#)

**Copy settings from existing bucket - optional**  
Only the bucket settings in the following configuration are copied.

[Choose bucket](#)

Format: s3://bucket/prefix

**Object Ownership** [Info](#)

- Save screenshot as:

**Bucket Versioning**

Versioning is a means of keeping multiple variants of an object in the same bucket. You can use versioning to preserve, retrieve, and restore every version of every object stored in your Amazon S3 bucket. With versioning, you can easily recover from both unintended user actions and application failures. [Learn more](#) [u](#)

**Bucket Versioning**

☐ Disable  
☒ Enable

**Tags - optional**  
You can use bucket tags to analyze, manage and specify permissions for a bucket. [Learn more](#) [u](#)

[i](#) You can use s3:ListTagsForResource, s3:TagResource, and s3:UntagResource APIs to manage tags on S3 general purpose buckets for access control in addition to cost allocation and resource organization. To ensure a seamless transition, please provide permissions to s3:ListTagsForResource, s3:TagResource, and s3:UntagResource actions. [Learn more](#) [u](#)

No tags associated with this bucket

– S3 bucket with versioning enabled.

2. Update main.tf to add S3 backend configuration:

Add this at the beginning of main.tf (before the provider block):

```
terraform {
  backend "s3" {
    bucket = "myapp-s3-bucket-demo"
    key    = "myapp/terraform.tfstate"
    region = "me-central-1"
    encrypt = true
    use_lockfile = true
  }
}
```

- Save screenshot as:

```
@hamail-maker →~/Lab13 $ cat main.tf
provider "aws" {
  shared_config_files = ["~/.aws/config"]
  shared_credentials_files = ["~/.aws/credentials"]
}

terraform {
  backend "s3" {
    bucket = "myapp-s3-bucket-demo-first"
    key    = "myapp/terraform.tfstate"
    region = "me-central-1"
    encrypt = true
    use_lockfile = true
  }
}

resource "aws_iam_group" "developers" {
  name = "developers"
  path = "/groups/"
}

output "group_details" {
  value = {
    group_name = aws_iam_group.developers.name
    group_arn  = aws_iam_group.developers.arn
    unique_id  = aws_iam_group.developers.unique_id
  }
}
```

– main.tf showing backend configuration.

### 3. Reinitialize Terraform with the backend:

terraform init -migrate-state

- Type yes when prompted to migrate state
- **Save screenshot as:** task6\_terraform\_init\_migrate.png – terraform init output showing state migration.

```
@hamail-maker →~/Lab13 $ terraform init -migrate-state

Initializing the backend...
Do you want to copy existing state to the new backend?
Pre-existing state was found while migrating the previous "local" backend to the
newly configured "s3" backend. No existing state was found in the newly
configured "s3" backend. Do you want to copy this state to the new "s3"
backend? Enter "yes" to copy and "no" to start with an empty state.

Enter a value: yes

Successfully configured the backend "s3"! Terraform will automatically
use this backend unless the backend configuration changes.

Initializing provider plugins...
- Reusing previous version of hashicorp/aws from the dependency lock file
- Reusing previous version of hashicorp/null from the dependency lock file
- Using previously-installed hashicorp/aws v6.28.0
- Using previously-installed hashicorp/null v3.2.4

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
@hamail-maker →~/Lab13 $
```

### 4. Apply the configuration:

terraform apply -auto-approve -var="iam\_password=MySecurePass123!"

- **Save screenshot as:** task6\_terraform\_apply.png – terraform apply with remote state.

```
@hamill-maker →~/Lab13 $ terraform apply -auto-approve -var="iam_password=MySecurePass123!"
aws_iam_group.developers: Refreshing state... [id-developers]
aws_iam_user.lb: Refreshing state... [id-loadbalancer]
aws_iam_group_policy_attachment.developer_ec2_fullaccess: Refreshing state... [id-developers-20260110080811160100000002]
aws_iam_group_policy_attachment.change_password: Refreshing state... [id-developers-20260110080811142100000001]
null_resource.create_login_profile: Refreshing state... [id-8622112372841140320]
aws_iam_access_key.lb_access_key: Refreshing state... [id-AKIA42E3QACSJ0WPHY6M]
aws_iam_user_group_membership.lb_membership: Refreshing state... [id-terraform-20260110065739102600000001]

No changes. Your infrastructure matches the configuration.

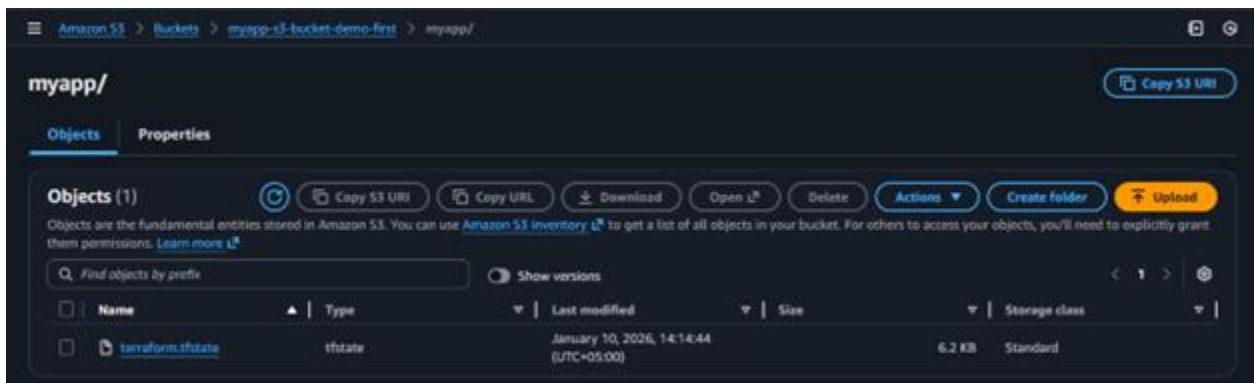
Terraform has compared your real infrastructure against your configuration and found no differences, so no changes are needed.

Apply complete! Resources: 0 added, 0 changed, 0 destroyed.

Outputs:
access_key_id = "AKIA42E3QACSJ0WPHY6M"
access_key_secret = <sensitive>
group_details = {
  "group_arn" = "arn:aws:iam::880794402980:group/groups/developers"
  "group_name" = "developers"
  "unique_id" = "AGPA42E3QACSJ0T12ECP"
}
user_details = {
  "unique_id" = "AIDM42E3QACSJ0WPHY6M"
  "user_arn" = "arn:aws:iam::880794402980:user/users/loadbalancer"
  "user_name" = "loadbalancer"
}
```

#### 5. Verify state file in S3:

- Navigate to S3 → myapp-s3-bucket-demo → myapp/
- You should see terraform.tfstate file
- **Save screenshot as:** task6\_s3\_tfstate\_file.png – S3 bucket showing terraform.tfstate file.



#### 6. Check local state file:

ls -la terraform.tfstate\*

- **Save screenshot as:** task6\_local\_state\_backup.png – showing local state backup file.

```
@hamill-maker →~/Lab13 $ ls -la terraform.tfstate*
-rw-rw-r-- 1 codespace codespace 0 Jan 10 09:14 terraform.tfstate
-rw-rw-r-- 1 codespace codespace 6315 Jan 10 09:14 terraform.tfstate.backup
@hamill-maker →~/Lab13 $
```

#### 7. Destroy resources and verify state change:

terraform destroy -auto-approve

- **Save screenshot as:** task6\_terraform\_destroy.png – terraform destroy output.

```

null_resource.create_login_profile: Destroying... [id-8622112372841140320]
null_resource.create_login_profile: Destruction complete after 0s
aws_iam_group_policy_attachment.developer_ec2_fullaccess: Destroying... [id-developers-20260110080811160100000002]
aws_iam_group_policy_attachment.change_password: Destroying... [id-developers-20260110080811142100000001]
aws_iam_user_group_membership.lb_membership: Destroying... [id-terraform-20260110065739102600000001]
aws_iam_access_key.lb_access_key: Destroying... [id-AKIA42E3QACS703M4WGM]
aws_iam_group_policy_attachment.developer_ec2_fullaccess: Destruction complete after 1s
aws_iam_user_group_membership.lb_membership: Destruction complete after 1s
aws_iam_group_policy_attachment.change_password: Destruction complete after 1s
aws_iam_group.developers: Destroying... [id-developers]
aws_iam_access_key.lb_access_key: Destruction complete after 1s
aws_iam_user.lb: Destroying... [id-loadbalancer]
aws_iam_group.developers: Destruction complete after 0s
aws_iam_user.lb: Destruction complete after 3s

Destroy complete! Resources: 7 destroyed.

```

8. Verify updated state in S3:

- Refresh S3 bucket view
- Check the terraform.tfstate file (it should show empty resources)
- **Save screenshot as:** task6\_s3\_tfstate\_destroyed.png – S3 state file after destroy.

```

Pretty print
{
  "version": 4,
  "terraform_version": "1.6.3",
  "serial": 2,
  "lineage": "6f41a236-879f-ae5a-da4a-4b53f66fd66a",
  "outputs": {},
  "resources": [],
  "check_results": null
}

```

## Task 7 – Create Multiple Users from CSV File

In this task, you will create multiple IAM users dynamically from a CSV file.

1. Create locals.tf file:

```

locals {
  users = csvdecode(file("users.csv"))
}

```

- **Save screenshot as:** task7\_locals\_tf.png – content of locals.tf file.

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
GNU nano 7.2 locals.tf *
locals {
  users = csvdecode(file("users.csv"))
}

```

2. Create users.csv file:

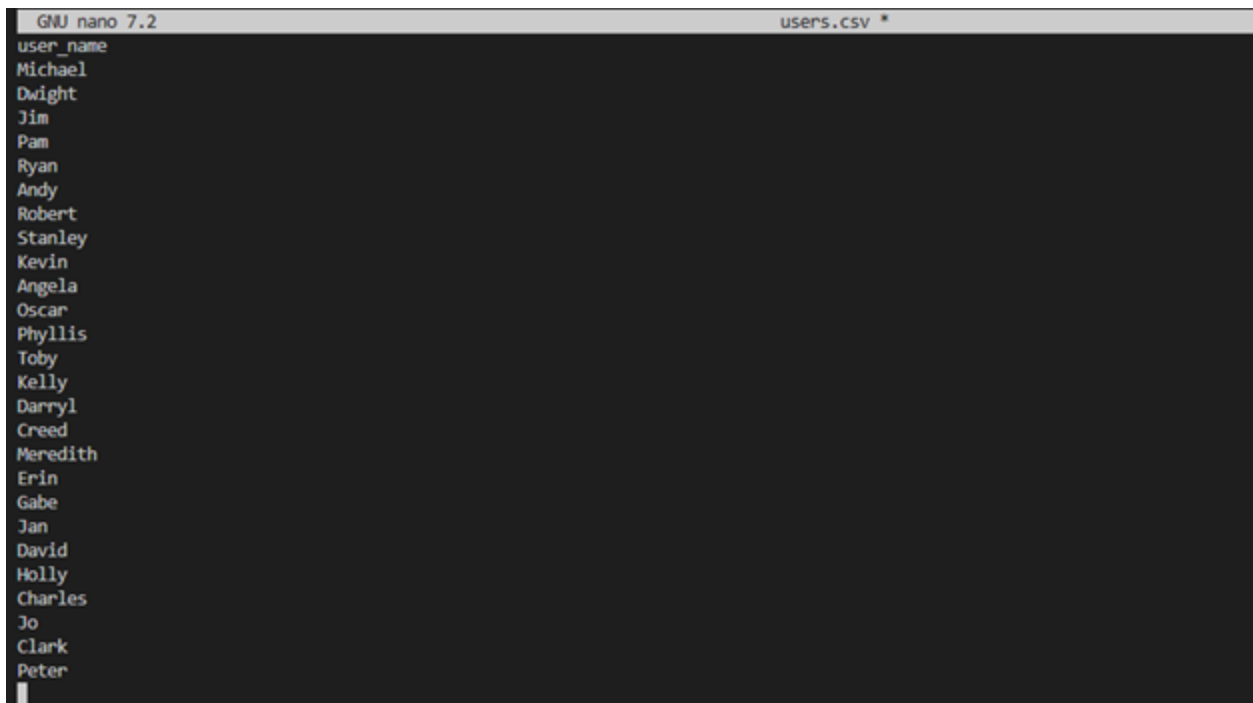
```

user_name
Michael
Dwight
Jim
Pam
Ryan
Andy
Robert
Stanley
Kevin
Angela

```

Oscar  
Phyllis  
Toby  
Kelly  
Darryl  
Creed  
Meredith  
Erin  
Gabe  
Jan  
David  
Holly  
Charles  
Jo  
Clark  
Peter

- **Save screenshot as:** task7\_users\_csv.png – content of users.csv file.



```
GNU nano 7.2 users.csv *
user_name
Michael
Dwight
Jim
Pam
Ryan
Andy
Robert
Stanley
Kevin
Angela
Oscar
Phyllis
Toby
Kelly
Darryl
Creed
Meredith
Erin
Gabe
Jan
David
Holly
Charles
Jo
Clark
Peter
```

### 3. Update main.tf to create multiple users:

Replace the single user resources with:

# Create multiple IAM users from CSV

```
resource "aws_iam_user" "users" {
  for_each = { for user in local.users : user.user_name => user }
```

```
    name      = each.value.user_name
```

```
    path      = "/users/"
```

```
    force_destroy = true
```

```
    tags = {
```

```
      DisplayName = each.value.user_name
```

```
      CreatedBy   = "Terraform"
```

```
    }
```

```
  }
```

```

# Add all users to developers group
resource "aws_iam_user_group_membership" "users_membership" {
  for_each = aws_iam_user.users

  user = each.value.name
  groups = [
    aws_iam_group.developers.name
  ]
}

# Create login profiles for all users
resource "null_resource" "create_login_profiles" {
  for_each = aws_iam_user.users

  triggers = {
    password_hash = sha256(var.iam_password)
    user          = each.value.name
  }

  depends_on = [aws_iam_user.users]

  provisioner "local-exec" {
    command = "${path.module}/create-login-profile.sh ${each.value.name}
    ${var.iam_password}"
  }
}

# Create access keys for all users
resource "aws_iam_access_key" "users_access_keys" {
  for_each = aws_iam_user.users

  user = each.value.name
}

# Output all user details
output "all_users_details" {
  value = {
    for user_name, user in aws_iam_user.users : user_name => {
      user_arn      = user.arn
      user_unique_id = user.unique_id
      access_key_id = aws_iam_access_key.users_access_keys[user_name].id
    }
  }
}

# Output all access key secrets (sensitive)
output "all_access_key_secrets" {
  value = {
    for user_name, key in aws_iam_access_key.users_access_keys : user_name => key.secret
  }
  sensitive = true
}

```

- **Save screenshot as:** task7\_main\_tf\_multiple\_users.png – main.tf showing multiple user resources.



```
# --- Task 7: Multiple Users from CSV ---
resource "aws_iam_user" "users" {
  for_each = { for user in local.users : user.user_name => user }

  name           = each.value.user_name
  path           = "/users/"
  force_destroy = true

  tags = {
    DisplayName = each.value.user_name
    CreatedBy   = "Terraform"
  }
}

resource "aws_iam_user_group_membership" "users_membership" {
  for_each = aws_iam_user.users

  user       = each.value.name
  groups     = [aws_iam_group.developers.name]
}

resource "null_resource" "create_login_profiles" {
  for_each = aws_iam_user.users

  triggers = {
    password_hash = sha256(var.iam_password)
    user          = each.value.name
  }
}
```

4. Reinitialize Terraform (since we changed the configuration significantly):

terraform init

- Save screenshot

as:

```
@hamail-maker →~/Lab13 $ terraform init

Initializing the backend...

Initializing provider plugins...
- Reusing previous version of hashicorp/aws from the dependency lock file
- Reusing previous version of hashicorp/null from the dependency lock file
- Using previously-installed hashicorp/null v3.2.4
- Using previously-installed hashicorp/aws v6.28.0

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
@hamail-maker →~/Lab13 $
```

– terraform init output.

5. Apply the configuration to create all users:

terraform apply -auto-approve -var="iam\_password=MySecurePass123!"

- Save screenshot as: task7\_terraform\_apply.png – terraform apply output showing all users being created.

```

"Robert" = {
  "access_key_id" = "AKIA42E3QACSKKH5ND7L"
  "user_arn" = "arn:aws:iam::880794402980:user/users/Robert"
  "user_unique_id" = "AIDA42E3QACSKAFLACDGI"
}
"Ryan" = {
  "access_key_id" = "AKIA42E3QACSNOSVM4NX"
  "user_arn" = "arn:aws:iam::880794402980:user/users/Ryan"
  "user_unique_id" = "AIDA42E3QACSNIQNLT43"
}
"Stanley" = {
  "access_key_id" = "AKIA42E3QACSCJTEBYNK"
  "user_arn" = "arn:aws:iam::880794402980:user/users/Stanley"
  "user_unique_id" = "AIDA42E3QACSG4GVYNLBM"
}
"Toby" = {
  "access_key_id" = "AKIA42E3QACSJLHZH3MW"
  "user_arn" = "arn:aws:iam::880794402980:user/users/Toby"
  "user_unique_id" = "AIDA42E3QACSJGSGHXGGZ"
}
}
group_details = {
  "group_arn" = "arn:aws:iam::880794402980:group/groups/developers"
  "group_name" = "developers"
  "unique_id" = "AGPA42E3QACSFZ55LN52J"
}
user_details = {
  "unique_id" = "AIDA42E3QACSJZRZSW55K"
  "user_arn" = "arn:aws:iam::880794402980:user/users/loadbalancer"
  "user_name" = "loadbalancer"
}
}

```

6. Display the outputs:

terraform output

- **Save screenshot as:** task7\_terraform\_output.png – terraform output showing all user details.

```

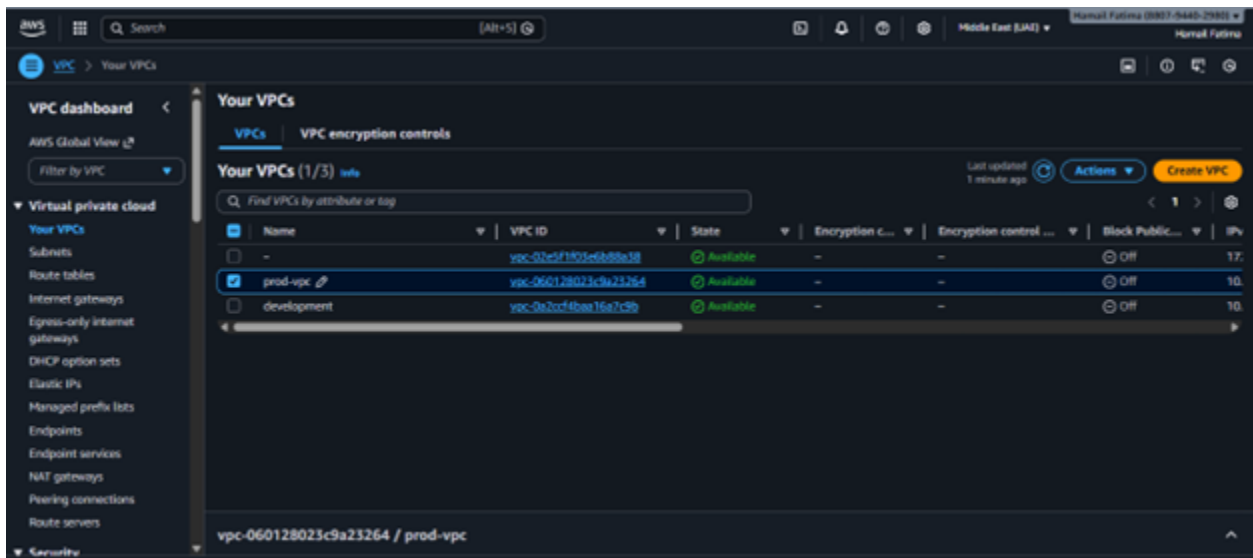
@Hamail-maker → ~/Lab13 $ terraform output
access_key_id = "AKIA42E3QACSFJZ7YNJ"
access_key_secret = <sensitive>
all_access_key_secrets = <sensitive>
all_users_details = {
  "Andy" = {
    "access_key_id" = "AKIA42E3QACSG4JFISPJ"
    "user_arn" = "arn:aws:iam::880794402980:user/users/Andy"
    "user_unique_id" = "AIDA42E3QACSJNCLJ2SWB"
  }
  "Angela" = {
    "access_key_id" = "AKIA42E3QACS0JR4QH7H"
    "user_arn" = "arn:aws:iam::880794402980:user/users/Angela"
    "user_unique_id" = "AIDA42E3QACSN2TZWF5WV"
  }
  "Charles" = {
    "access_key_id" = "AKIA42E3QACSPJFUJBPL"
    "user_arn" = "arn:aws:iam::880794402980:user/users/Charles"
    "user_unique_id" = "AIDA42E3QACSA2ISRTY"
  }
  "Clark" = {
    "access_key_id" = "AKIA42E3QACS0DTP3PMQ"
    "user_arn" = "arn:aws:iam::880794402980:user/users/Clark"
    "user_unique_id" = "AIDA42E3QACSELJGFU3TQ"
  }
}

```

7. View secrets in terraform. tfstate:

cat terraform.tfstate | grep -A 5 "all\_access\_key\_secrets"

- **Save screenshot as:** task7\_tfstate\_secrets.png – terraform.tfstate showing access key secrets.



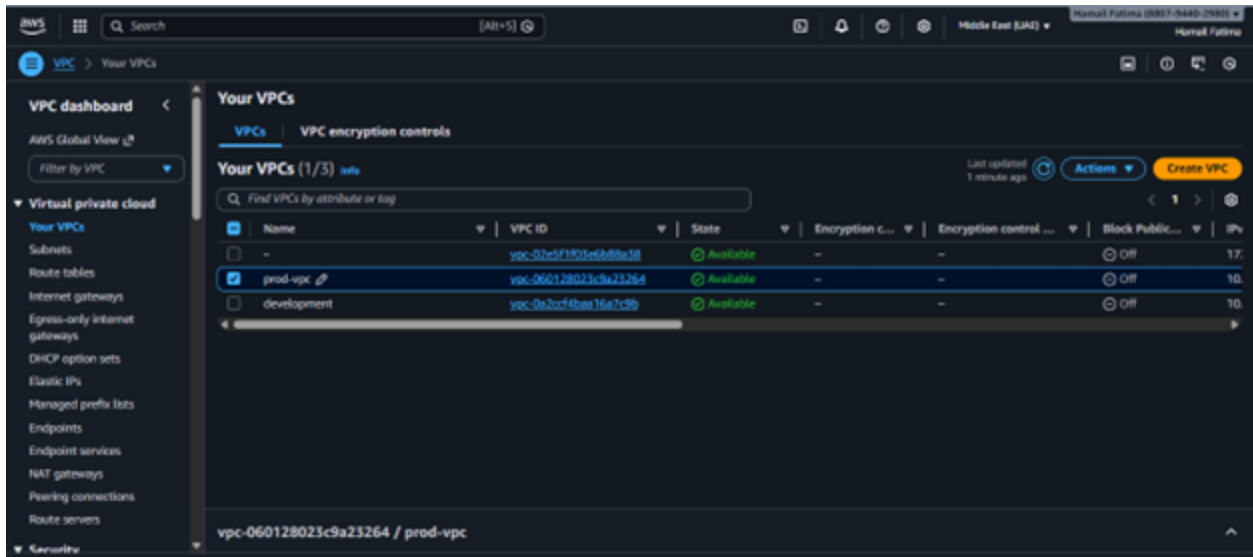
#### 8. Verify all users in AWS Console:

- Navigate to IAM → Users
- **Save screenshot as:** task7\_aws\_console\_all\_users.png – AWS Console showing all created users.

| <input type="checkbox"/> | User name               | Path    | Groups | Last activity | MFA | Password age | Consc |
|--------------------------|-------------------------|---------|--------|---------------|-----|--------------|-------|
| <input type="checkbox"/> | <a href="#">Andy</a>    | /users/ | 1      | -             | -   | ✓ 1 hour     | -     |
| <input type="checkbox"/> | <a href="#">Angela</a>  | /users/ | 1      | -             | -   | ✓ 1 hour     | -     |
| <input type="checkbox"/> | <a href="#">Charles</a> | /users/ | 1      | -             | -   | ✓ 1 hour     | -     |
| <input type="checkbox"/> | <a href="#">Clark</a>   | /users/ | 1      | -             | -   | ✓ 1 hour     | -     |
| <input type="checkbox"/> | <a href="#">Creed</a>   | /users/ | 1      | -             | -   | ✓ 1 hour     | -     |
| <input type="checkbox"/> | <a href="#">Darryl</a>  | /users/ | 1      | -             | -   | ✓ 1 hour     | -     |
| <input type="checkbox"/> | <a href="#">David</a>   | /users/ | 1      | -             | -   | ✓ 1 hour     | -     |

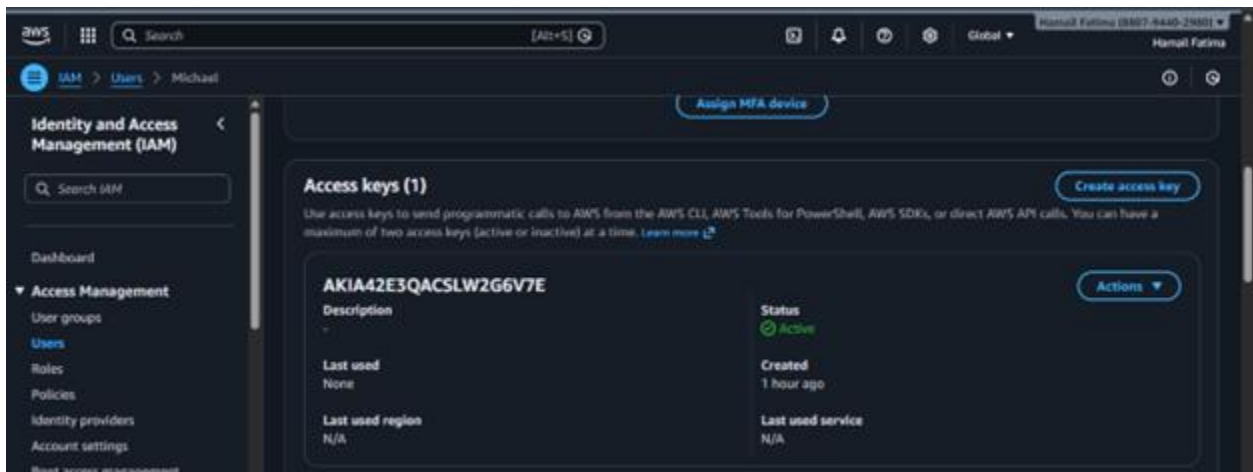
#### 9. Verify group membership:

- Navigate to IAM → Groups → developers → Users tab
- **Save screenshot as:** task7\_aws\_console\_group\_members.png – AWS Console showing all users in developers group.



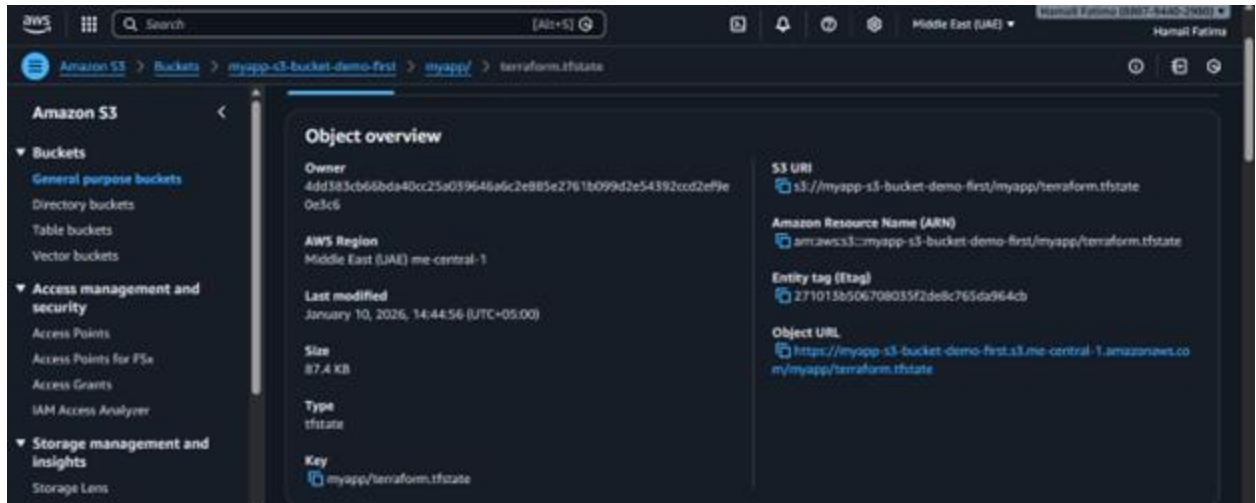
10. Verify one user's access keys:

- Click on any user (e.g., "Michael")
- Go to Security credentials tab
- **Save screenshot as:** task7\_aws\_console\_user\_access\_key.png – AWS Console showing user's access key.



11. Check terraform state in S3:

- Navigate to S3 bucket and view the state file
- **Save screenshot as:** task7\_s3\_tfstate\_multiple\_users.png – S3 showing updated state file.



## Cleanup

1. Destroy all resources:

terraform destroy -auto-approve

- **Save screenshot as:** cleanup\_destroy\_complete.png – terraform destroy completion output.

```
@Hamail-maker →/workspaces/CC_Hamail-Fatima_2023-BSE-023_Lab13 (main) $ terraform destroy -auto-approve -lock=false
var.iam_password
Password for all IAM users

Enter a value:

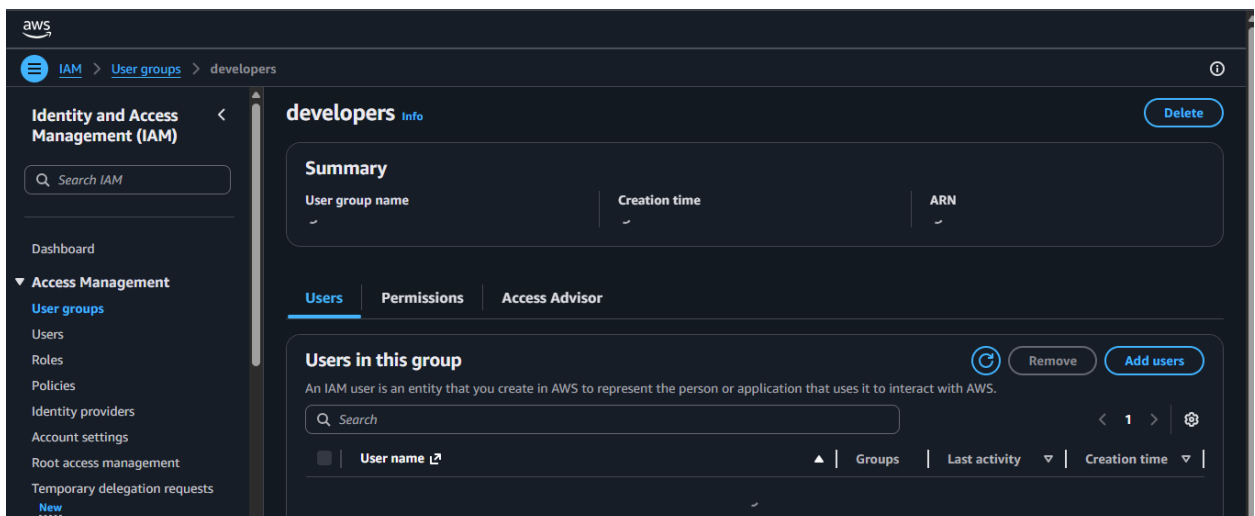
No changes. No objects need to be destroyed.

Either you have not created any objects yet or the existing objects were already deleted outside of Terraform.

Destroy complete! Resources: 0 destroyed.
@Hamail-maker →/workspaces/CC_Hamail-Fatima_2023-BSE-023_Lab13 (main) $
```

2. Verify users deleted in AWS Console:

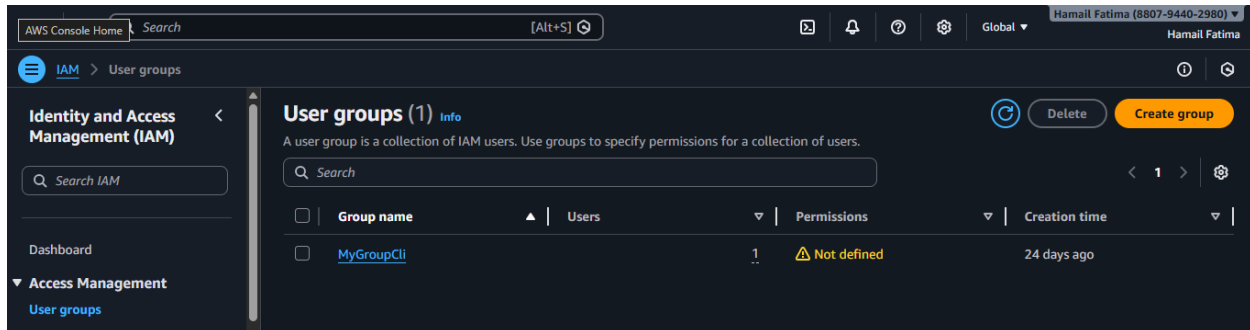
- Navigate to IAM → Users
- **Save screenshot as:** cleanup\_aws\_console\_users\_deleted.png – AWS Console showing no users.



3. Verify group deleted in AWS Console:

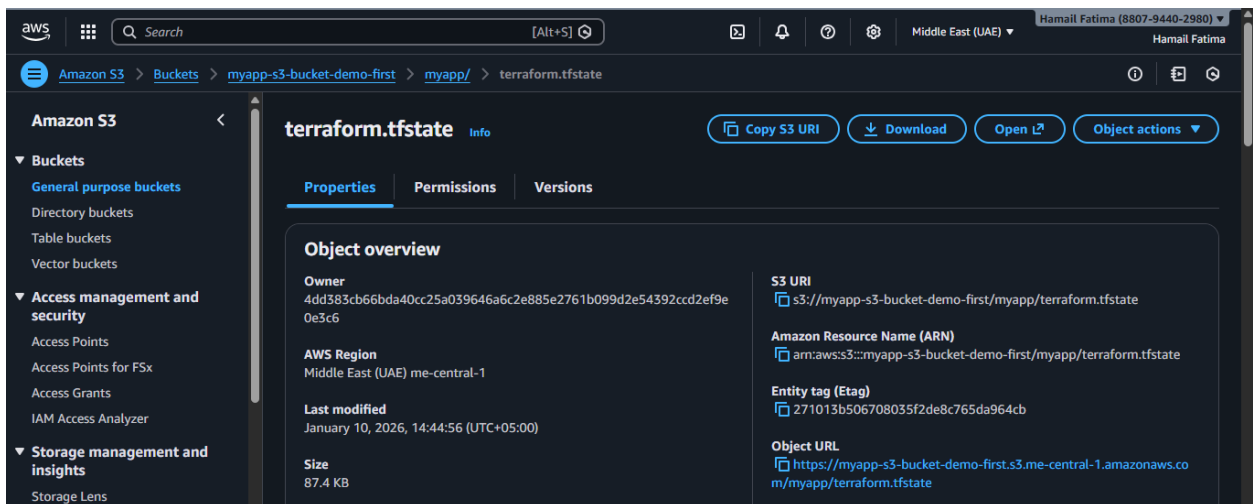
- Navigate to IAM → Groups
- **Save screenshot as:** cleanup\_aws\_console\_group\_deleted.png – AWS Console showing

developers group deleted.



4. Check S3 state file:

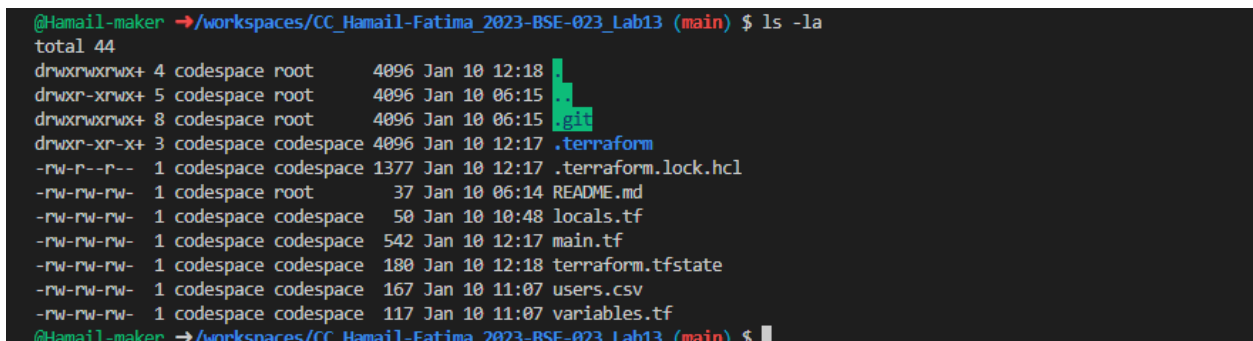
- Navigate to S3 bucket
- **Save screenshot as:** cleanup\_s3\_empty\_state.png – S3 showing empty state.



5. List all project files:

ls -la

- **Save screenshot as:** cleanup\_final\_files.png – showing final project structure.



6. (Optional) Delete S3 bucket:

- If you want to clean up completely, delete the S3 bucket from AWS Console
- **Save screenshot as:** cleanup\_s3\_bucket\_deleted.png – S3 bucket deletion confirmation.